
第四部分：泛型编程概念

4.1 什么是迭代器？

迭代器就像是一个“智能指针”，它提供了一种统一的方式来遍历不同类型的容器。想象一下，无论是数组、链表还是树结构，您都可以用同样的方式来访问其中的元素。

迭代器的基本使用

```
cpp

#include <vector>
#include <list>
#include <iostream>
using namespace std;

int main() {
    vector<int> vec{1, 2, 3, 4, 5};
    list<int> lst{1, 2, 3, 4, 5};

    // vector的迭代器
    cout << "vector内容: ";
    for (auto it = vec.begin(); it != vec.end(); ++it) {
        cout << *it << " "; // 解引用获取值
    }
    cout << endl;

    // list的迭代器（使用方式相同！）
    cout << "list内容: ";
    for (auto it = lst.begin(); it != lst.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // 反向迭代器
    cout << "vector反向: ";
    for (auto it = vec.rbegin(); it != vec.rend(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    return 0;
}
```

迭代器的类型

STL定义了5种迭代器类型，从弱到强：

1. **输入迭代器 (Input Iterator)**：只读，单向
2. **输出迭代器 (Output Iterator)**：只写，单向
3. **前向迭代器 (Forward Iterator)**：读写，单向，可多次遍历
4. **双向迭代器 (Bidirectional Iterator)**：前向+后向
5. **随机访问迭代器 (Random Access Iterator)**：支持跳跃访问

```
cpp

#include <vector>
#include <list>
#include <iostream>
using namespace std;

int main() {
    vector<int> vec{10, 20, 30, 40, 50};
    list<int> lst{10, 20, 30, 40, 50};

    // vector支持随机访问迭代器
    auto vec_it = vec.begin();
    vec_it += 2; // 跳到第3个元素
    cout << "vector第3个元素: " << *vec_it << endl;

    // list只支持双向迭代器，不能跳跃访问
    auto lst_it = lst.begin();
    advance(lst_it, 2); // 使用advance函数移动
    cout << "list第3个元素: " << *lst_it << endl;

    // 计算距离
    cout << "vector中的距离: " << distance(vec.begin(), vec.end()) << endl;
    cout << "list中的距离: " << distance(lst.begin(), lst.end()) << endl;

    return 0;
}
```

4.2 算法库

STL提供了大量的通用算法，这些算法可以与任何支持相应迭代器的容器配合使用。

常用查找算法

```
cpp
```

```

#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

int main() {
    vector<int> numbers{3, 1, 4, 1, 5, 9, 2, 6};

    // find: 查找元素
    auto it = find(numbers.begin(), numbers.end(), 5);
    if (it != numbers.end()) {
        cout << "找到5, 位置: " << distance(numbers.begin(), it) << endl;
    }

    // find_if: 按条件查找
    auto it2 = find_if(numbers.begin(), numbers.end(),
        [](int x) { return x > 5; });
    if (it2 != numbers.end()) {
        cout << "第一个大于5的数: " << *it2 << endl;
    }

    // count: 计数
    int count_1 = count(numbers.begin(), numbers.end(), 1);
    cout << "数字1出现了 " << count_1 << " 次" << endl;

    // count_if: 按条件计数
    int count_even = count_if(numbers.begin(), numbers.end(),
        [](int x) { return x % 2 == 0; });
    cout << "偶数有 " << count_even << " 个" << endl;

    return 0;
}

```

常用修改算法

cpp

```

#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

void printVector(const vector<int>& vec, const string& title) {
    cout << title << ": ";
    for (int x : vec) cout << x << " ";
    cout << endl;
}

int main() {
    vector<int> numbers{3, 1, 4, 1, 5, 9, 2, 6};
    printVector(numbers, "原始数据");

    // sort: 排序
    vector<int> sorted = numbers;
    sort(sorted.begin(), sorted.end());
    printVector(sorted, "升序排序");

    // 降序排序
    sort(sorted.begin(), sorted.end(), greater<int>());
    printVector(sorted, "降序排序");

    // reverse: 反转
    vector<int> reversed = numbers;
    reverse(reversed.begin(), reversed.end());
    printVector(reversed, "反转");

    // unique: 去重 (需要先排序)
    vector<int> unique_nums = numbers;
    sort(unique_nums.begin(), unique_nums.end());
    auto new_end = unique(unique_nums.begin(), unique_nums.end());
    unique_nums.erase(new_end, unique_nums.end());
    printVector(unique_nums, "去重");

    // transform: 变换
    vector<int> transformed = numbers;
    transform(transformed.begin(), transformed.end(), transformed.begin(),
        [](int x) { return x * 2; });
    printVector(transformed, "每个元素*2");

    return 0;
}

```

cpp

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

int main() {
    vector<int> source{1, 2, 3, 4, 5};
    vector<int> dest(5);

    // copy: 复制
    copy(source.begin(), source.end(), dest.begin());
    cout << "复制后dest: ";
    for (int x : dest) cout << x << " ";
    cout << endl;

    // copy_if: 按条件复制
    vector<int> even_nums;
    copy_if(source.begin(), source.end(), back_inserter(even_nums),
        [](int x) { return x % 2 == 0; });
    cout << "偶数: ";
    for (int x : even_nums) cout << x << " ";
    cout << endl;

    // fill: 填充
    vector<int> filled(5);
    fill(filled.begin(), filled.end(), 42);
    cout << "填充42: ";
    for (int x : filled) cout << x << " ";
    cout << endl;

    // generate: 生成
    vector<int> generated(5);
    int counter = 0;
    generate(generated.begin(), generated.end(),
        [&counter]() { return ++counter * 10; });
    cout << "生成数列: ";
    for (int x : generated) cout << x << " ";
    cout << endl;

    return 0;
}
```

4.3 函数对象（仿函数）

函数对象是重载了 `operator()` 的类，可以像函数一样被调用，但比普通函数更灵活。

自定义函数对象

```
cpp
```

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

// 简单的函数对象
class MultiplyBy {
private:
    int factor;
public:
    MultiplyBy(int f) : factor(f) {}

    int operator()(int x) const {
        return x * factor;
    }
};

// 更复杂的函数对象：累加器
class Accumulator {
private:
    int sum;
public:
    Accumulator() : sum(0) {}

    void operator()(int x) {
        sum += x;
    }

    int getSum() const { return sum; }
};

int main() {
    vector<int> numbers{1, 2, 3, 4, 5};

    // 使用MultiplyBy
    MultiplyBy multiply3(3);
    vector<int> tripled(5);
    transform(numbers.begin(), numbers.end(), tripled.begin(), multiply3);

    cout << "原数组: ";
    for (int x : numbers) cout << x << " ";
    cout << endl;

    cout << "乘以3: ";
    for (int x : tripled) cout << x << " ";
    cout << endl;
}
```

```
// 使用Accumulator
Accumulator acc = for_each(numbers.begin(), numbers.end(), Accumulator());
cout << "总和: " << acc.getSum() << endl;

return 0;
}
```

预定义函数对象

STL提供了许多预定义的函数对象：

cpp


```

#include <vector>
#include <algorithm>
#include <functional>
#include <iostream>
using namespace std;

int main() {
    vector<int> numbers{3, 1, 4, 1, 5, 9, 2, 6};

    // 使用预定义函数对象排序
    sort(numbers.begin(), numbers.end(), greater<int>());
    cout << "降序排序: ";
    for (int x : numbers) cout << x << " ";
    cout << endl;

    // 使用函数适配器
    vector<int> vec1{1, 2, 3, 4, 5};
    vector<int> vec2{2, 2, 2, 2, 2};
    vector<int> result(5);

    // 每个元素加2
    transform(vec1.begin(), vec1.end(), result.begin(),
        bind(plus<int>(), placeholders::_1, 2));
    cout << "每个元素+2: ";
    for (int x : result) cout << x << " ";
    cout << endl;

    // 两个数组相加
    transform(vec1.begin(), vec1.end(), vec2.begin(), result.begin(),
        plus<int>());
    cout << "数组相加: ";
    for (int x : result) cout << x << " ";
    cout << endl;

    return 0;
}

```

4.4 Lambda表达式 (C++11)

Lambda表达式是C++11引入的功能，让创建简单的函数对象变得非常容易：

cpp

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

int main() {
    vector<int> numbers{3, 1, 4, 1, 5, 9, 2, 6};

    // 基本lambda表达式
    auto isEven = [](int x) { return x % 2 == 0; };

    // 使用lambda进行查找
    auto it = find_if(numbers.begin(), numbers.end(), isEven);
    if (it != numbers.end()) {
        cout << "第一个偶数: " << *it << endl;
    }

    // 捕获外部变量
    int threshold = 5;
    auto countGreater = count_if(numbers.begin(), numbers.end(),
        [threshold](int x) { return x > threshold; });
    cout << "大于" << threshold << "的数有: " << countGreater << " 个" << endl;

    // 修改捕获的变量 (需要mutable)
    int sum = 0;
    for_each(numbers.begin(), numbers.end(),
        [&sum](int x) mutable { sum += x; });
    cout << "总和: " << sum << endl;

    // 复杂的lambda: 排序
    vector<string> words{"apple", "banana", "cherry", "date"};
    sort(words.begin(), words.end(),
        [](const string& a, const string& b) {
            return a.length() < b.length(); // 按长度排序
        });

    cout << "按长度排序: ";
    for (const auto& word : words) {
        cout << word << " ";
    }
    cout << endl;

    return 0;
}
```

4.5 实战示例：数据分析工具

让我们创建一个综合使用算法和函数对象的数据分析工具：

```
cpp
```

```
#include <vector>
#include <algorithm>
#include <numeric>
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;

class DataAnalyzer {
private:
    vector<double> data;

public:
    // 添加数据
    void addData(const vector<double>& newData) {
        data.insert(data.end(), newData.begin(), newData.end());
    }

    // 基础统计
    double mean() const {
        if (data.empty()) return 0.0;
        return accumulate(data.begin(), data.end(), 0.0) / data.size();
    }

    double median() const {
        if (data.empty()) return 0.0;

        vector<double> sorted_data = data;
        sort(sorted_data.begin(), sorted_data.end());

        size_t n = sorted_data.size();
        if (n % 2 == 0) {
            return (sorted_data[n/2 - 1] + sorted_data[n/2]) / 2.0;
        } else {
            return sorted_data[n/2];
        }
    }

    double standardDeviation() const {
        if (data.size() <= 1) return 0.0;

        double avg = mean();
        double sum_squared_diff = 0.0;

        for_each(data.begin(), data.end(),
            [&](double x) {
```

```

        double diff = x - avg;
        sum_squared_diff += diff * diff;
    });

    return sqrt(sum_squared_diff / (data.size() - 1));
}

pair<double, double> minMax() const {
    if (data.empty()) return {0.0, 0.0};

    auto minmax_pair = minmax_element(data.begin(), data.end());
    return {*minmax_pair.first, *minmax_pair.second};
}

// 数据筛选
vector<double> filterByRange(double min_val, double max_val) const {
    vector<double> filtered;
    copy_if(data.begin(), data.end(), back_inserter(filtered),
        [min_val, max_val](double x) {
            return x >= min_val && x <= max_val;
        });
    return filtered;
}

// 数据变换
void transform(function<double(double)> func) {
    std::transform(data.begin(), data.end(), data.begin(), func);
}

// 查找异常值（基于标准差）
vector<double> findOutliers(double threshold = 2.0) const {
    double avg = mean();
    double std_dev = standardDeviation();
    vector<double> outliers;

    copy_if(data.begin(), data.end(), back_inserter(outliers),
        [avg, std_dev, threshold](double x) {
            return abs(x - avg) > threshold * std_dev;
        });

    return outliers;
}

// 数据分组统计
map<string, int> categorize(function<string(double)> classifier) const {
    map<string, int> categories;

```

```

    for_each(data.begin(), data.end(),
        [&](double x) {
            string category = classifier(x);
            categories[category]++;
        });

    return categories;
}

// 显示统计摘要
void printSummary() const {
    if (data.empty()) {
        cout << "没有数据" << endl;
        return;
    }

    cout << fixed << setprecision(2);
    cout << "=== 数据统计摘要 ===" << endl;
    cout << "数据点数: " << data.size() << endl;
    cout << "平均值: " << mean() << endl;
    cout << "中位数: " << median() << endl;
    cout << "标准差: " << standardDeviation() << endl;

    auto minmax = minMax();
    cout << "最小值: " << minmax.first << endl;
    cout << "最大值: " << minmax.second << endl;

    auto outliers = findOutliers();
    cout << "异常值数量: " << outliers.size() << endl;
    if (!outliers.empty()) {
        cout << "异常值: ";
        for (double x : outliers) {
            cout << x << " ";
        }
        cout << endl;
    }
}

```

// 获取数据副本

```

vector<double> getData() const { return data; }
};

```

int main() {

DataAnalyzer analyzer;

// 添加一些测试数据

```

vector<double> dataset1{85.5, 92.3, 78.1, 95.7, 88.2, 91.4, 83.6, 89.8};

```

```

vector<double> dataset2{76.3, 94.2, 87.5, 120.5, 82.1, 90.7, 85.3}; // 包含异常值

analyzer.addData(dataset1);
analyzer.addData(dataset2);

// 显示基础统计
analyzer.printSummary();

// 筛选数据
auto normal_range = analyzer.filterByRange(70.0, 100.0);
cout << "\n70-100范围内的数据点: " << normal_range.size() << " 个" << endl;

// 数据分类
auto categories = analyzer.categorize([](double x) -> string {
    if (x >= 90) return "优秀";
    else if (x >= 80) return "良好";
    else if (x >= 70) return "及格";
    else return "不及格";
});

cout << "\n=== 成绩分布 ===" << endl;
for (const auto& pair : categories) {
    cout << pair.first << ": " << pair.second << " 人" << endl;
}

// 数据变换示例
DataAnalyzer normalizer;
normalizer.addData(analyzer.getData());

// 标准化数据（转换为z-score）
double mean_val = normalizer.mean();
double std_val = normalizer.standardDeviation();

normalizer.transform([mean_val, std_val](double x) {
    return (x - mean_val) / std_val;
});

cout << "\n=== 标准化后的数据统计 ===" << endl;
normalizer.printSummary();

return 0;
}

```

第五部分：高级STL特性

5.1 容器适配器

容器适配器是基于其他容器实现的特殊容器，它们提供了特定的接口。

stack（栈）

```
cpp

#include <stack>
#include <iostream>
using namespace std;

int main() {
    stack<int> st;

    // 入栈
    st.push(10);
    st.push(20);
    st.push(30);

    cout << "栈的大小: " << st.size() << endl;

    // 出栈
    while (!st.empty()) {
        cout << "栈顶元素: " << st.top() << endl;
        st.pop(); // 注意: pop() 不返回元素
    }

    return 0;
}
```

queue（队列）

```
cpp
```



```
#include <queue>
#include <iostream>
using namespace std;

int main() {
    queue<string> q;

    // 入队
    q.push("第一个");
    q.push("第二个");
    q.push("第三个");

    cout << "队列大小: " << q.size() << endl;

    // 出队
    while (!q.empty()) {
        cout << "队首元素: " << q.front() << endl;
        cout << "队尾元素: " << q.back() << endl;
        q.pop();
        cout << "---" << endl;
    }

    return 0;
}
```

priority_queue（优先队列）

cpp

```

#include <queue>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    // 默认是最大堆
    priority_queue<int> max_heap;

    max_heap.push(3);
    max_heap.push(1);
    max_heap.push(4);
    max_heap.push(1);
    max_heap.push(5);

    cout << "最大堆: ";
    while (!max_heap.empty()) {
        cout << max_heap.top() << " ";
        max_heap.pop();
    }
    cout << endl;

    // 最小堆
    priority_queue<int, vector<int>, greater<int>> min_heap;

    min_heap.push(3);
    min_heap.push(1);
    min_heap.push(4);
    min_heap.push(1);
    min_heap.push(5);

    cout << "最小堆: ";
    while (!min_heap.empty()) {
        cout << min_heap.top() << " ";
        min_heap.pop();
    }
    cout << endl;

    return 0;
}

```

5.2 关联容器详解

multimap 和 multiset

这些容器允许重复的键：

cpp

```
#include <map>
#include <set>
#include <iostream>
using namespace std;

int main() {
    // multiset: 允许重复元素的集合
    multiset<int> ms{3, 1, 4, 1, 5, 9, 2, 6, 5};

    cout << "multiset内容: ";
    for (int x : ms) {
        cout << x << " ";
    }
    cout << endl;

    cout << "数字1出现次数: " << ms.count(1) << endl;
    cout << "数字5出现次数: " << ms.count(5) << endl;

    // multimap: 允许重复键的映射
    multimap<string, int> scores;
    scores.insert({"Alice", 85});
    scores.insert({"Bob", 92});
    scores.insert({"Alice", 78}); // 同一个键的多个值
    scores.insert({"Bob", 88});

    cout << "\n所有成绩:" << endl;
    for (const auto& pair : scores) {
        cout << pair.first << ": " << pair.second << endl;
    }

    // 查找特定键的所有值
    cout << "\nAlice的所有成绩:" << endl;
    auto range = scores.equal_range("Alice");
    for (auto it = range.first; it != range.second; ++it) {
        cout << it->second << " ";
    }
    cout << endl;

    return 0;
}
```

5.3 无序容器 (C++11)

无序容器基于哈希表实现，提供平均 $O(1)$ 的查找时间：


```
#include <unordered_map>
#include <unordered_set>
#include <iostream>
#include <string>
using namespace std;

int main() {
    // unordered_set
    unordered_set<string> words{"apple", "banana", "cherry", "date"};

    // 查找
    if (words.find("banana") != words.end()) {
        cout << "找到banana" << endl;
    }

    // 插入
    words.insert("elderberry");

    cout << "unordered_set内容: ";
    for (const auto& word : words) {
        cout << word << " ";
    }
    cout << endl;

    // unordered_map: 单词计数器
    string text = "the quick brown fox jumps over the lazy dog the fox";
    unordered_map<string, int> word_count;

    // 简单的单词分割和计数
    string word;
    for (char c : text) {
        if (c == ' ') {
            if (!word.empty()) {
                word_count[word]++;
                word.clear();
            }
        } else {
            word += c;
        }
    }
    if (!word.empty()) {
        word_count[word]++;
    }

    cout << "\n单词计数:" << endl;
    for (const auto& pair : word_count) {
```

```
    cout << pair.first << ": " << pair.second << endl;
}

return 0;
}
```

5.4 实战示例：图书管理系统

让我们创建一个更复杂的系统，综合运用各种STL容器：

cpp

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <unordered_map>
#include <algorithm>
#include <queue>
using namespace std;

// 图书类
class Book {
public:
    string isbn;
    string title;
    string author;
    string category;
    int year;
    bool available;

    Book(const string& i, const string& t, const string& a,
         const string& c, int y)
        : isbn(i), title(t), author(a), category(c), year(y), available(true) {}

    void display() const {
        cout << "《" << title << "》 - " << author
              << " (" << year << ") [" << category << "]" << " "
              << (available ? "可借" : "已借出") << endl;
    }
};

// 借阅记录
struct BorrowRecord {
    string isbn;
    string borrower;
    string borrow_date;
    string return_date;

    BorrowRecord(const string& i, const string& b, const string& bd)
        : isbn(i), borrower(b), borrow_date(bd) {}
};

// 图书管理系统
class LibrarySystem {
private:
    vector<Book> books;                // 所有图书
```

```

map<string, size_t> isbn_to_index;           // ISBN到索引的映射
multimap<string, size_t> author_index;      // 作者索引
multimap<string, size_t> category_index;    // 分类索引
unordered_map<string, vector<size_t>> title_index; // 标题索引 (支持模糊搜索)

vector<BorrowRecord> borrow_history;       // 借阅历史
map<string, queue<string>> waiting_lists;   // 等待队列

```

public:

// 添加图书

```

void addBook(const string& isbn, const string& title,
             const string& author, const string& category, int year) {
    if (isbn_to_index.find(isbn) != isbn_to_index.end()) {
        cout << "图书已存在:" << isbn << endl;
        return;
    }

```

```

    books.emplace_back(isbn, title, author, category, year);
    size_t index = books.size() - 1;

```

// 更新各种索引

```

isbn_to_index[isbn] = index;
author_index.insert({author, index});
category_index.insert({category, index});

```

// 为标题创建关键词索引

```

vector<string> keywords = extractKeywords(title);
for (const string& keyword : keywords) {
    title_index[keyword].push_back(index);
}

```

```

    cout << "成功添加图书:" << title << endl;
}

```

// 提取标题关键词 (简单实现)

```

vector<string> extractKeywords(const string& title) {
    vector<string> keywords;
    string word;
    for (char c : title) {
        if (c == ' ' || c == '\t') {
            if (!word.empty()) {
                transform(word.begin(), word.end(), word.begin(), ::tolower);
                keywords.push_back(word);
                word.clear();
            }
        } else {
            word += c;
        }
    }
    return keywords;
}

```



```

    }
}
if (!word.empty()) {
    transform(word.begin(), word.end(), word.begin(), ::tolower);
    keywords.push_back(word);
}
return keywords;
}

```

// 按ISBN查找图书

```

Book* findBookByISBN(const string& isbn) {
    auto it = isbn_to_index.find(isbn);
    if (it != isbn_to_index.end()) {
        return &books[it->second];
    }
    return nullptr;
}

```

// 按作者查找图书

```

vector<Book*> findBooksByAuthor(const string& author) {
    vector<Book*> result;
    auto range = author_index.equal_range(author);
    for (auto it = range.first; it != range.second; ++it) {
        result.push_back(&books[it->second]);
    }
    return result;
}

```

// 按分类查找图书

```

vector<Book*> findBooksByCategory(const string& category) {
    vector<Book*> result;
    auto range = category_index.equal_range(category);
    for (auto it = range.first; it != range.second; ++it) {
        result.push_back(&books[it->second]);
    }
    return result;
}

```

// 按标题关键词搜索

```

vector<Book*> searchByTitle(const string& keyword) {
    vector<Book*> result;
    set<size_t> found_indices; // 避免重复

    string lower_keyword = keyword;
    transform(lower_keyword.begin(), lower_keyword.end(),
        lower_keyword.begin(), ::tolower);

```

```

auto it = title_index.find(lower_keyword);
if (it != title_index.end()) {
    for (size_t index : it->second) {
        if (found_indices.insert(index).second) {
            result.push_back(&books[index]);
        }
    }
}
return result;
}

```

// 借书

```

bool borrowBook(const string& isbn, const string& borrower) {
    Book* book = findBookByISBN(isbn);
    if (!book) {
        cout << "未找到图书: " << isbn << endl;
        return false;
    }

    if (!book->available) {
        cout << "图书已被借出，是否加入等待队列? (y/n): ";
        char choice;
        cin >> choice;
        if (choice == 'y' || choice == 'Y') {
            waiting_lists[isbn].push(borrower);
            cout << "已加入等待队列" << endl;
        }
        return false;
    }

    book->available = false;
    borrow_history.emplace_back(isbn, borrower, getCurrentDate());
    cout << "借书成功: " << book->title << endl;
    return true;
}

```

// 还书

```

bool returnBook(const string& isbn) {
    Book* book = findBookByISBN(isbn);
    if (!book) {
        cout << "未找到图书: " << isbn << endl;
        return false;
    }

    if (book->available) {
        cout << "该书未被借出" << endl;
        return false;
    }
}

```

```

}

book->available = true;

// 更新借阅记录
for (auto& record : borrow_history) {
    if (record.isbn == isbn && record.return_date.empty()) {
        record.return_date = getCurrentDate();
        break;
    }
}

// 检查等待队列
auto it = waiting_lists.find(isbn);
if (it != waiting_lists.end() && !it->second.empty()) {
    string next_borrower = it->second.front();
    it->second.pop();
    cout << "还书成功! 通知等待者 " << next_borrower
        << " 可以借阅 《" << book->title << "》 " << endl;
} else {
    cout << "还书成功: " << book->title << endl;
}

return true;
}

// 获取当前日期 (简化实现)
string getCurrentDate() {
    return "2024-01-01"; // 实际应用中应该获取真实日期
}

// 显示所有图书
void displayAllBooks() {
    cout << "\n=== 图书馆藏书 ===" << endl;
    for (const auto& book : books) {
        book.display();
    }
}

// 显示借阅统计
void displayBorrowStatistics() {
    cout << "\n=== 借阅统计 ===" << endl;

    // 统计每个人的借阅次数
    map<string, int> borrower_count;
    for (const auto& record : borrow_history) {
        borrower_count[record.borrower]++;
    }
}

```

```

}

cout << "借阅次数排行:" << endl;
vector<pair<string, int>> sorted_borrowers(borrower_count.begin(),
                                          borrower_count.end());
sort(sorted_borrowers.begin(), sorted_borrowers.end(),
      [](const pair<string, int>& a, const pair<string, int>& b) {
          return a.second > b.second;
      });

for (const auto& pair : sorted_borrowers) {
    cout << pair.first << ": " << pair.second << " 次" << endl;
}

// 统计最受欢迎的分类
map<string, int> category_count;
for (const auto& record : borrow_history) {
    Book* book = findBookByISBN(record.isbn);
    if (book) {
        category_count[book->category]++;
    }
}

cout << "\n最受欢迎的分类:" << endl;
for (const auto& pair : category_count) {
    cout << pair.first << ": " << pair.second << " 次借阅" << endl;
}

// 显示等待队列
void displayWaitingLists() {
    cout << "\n=== 等待队列 ===" << endl;
    for (const auto& pair : waiting_lists) {
        if (!pair.second.empty()) {
            Book* book = findBookByISBN(pair.first);
            if (book) {
                cout << "《" << book->title << "》 等待者: ";
                queue<string> temp = pair.second;
                while (!temp.empty()) {
                    cout << temp.front() << " ";
                    temp.pop();
                }
                cout << endl;
            }
        }
    }
}

```

```
};

int main() {
    LibrarySystem library;

    // 添加一些示例图书
    library.addBook("978-0-123456-78-9", "C++程序设计", "Bjarne Stroustrup", "编程", 2013);
    library.addBook("978-0-234567-89-0", "算法导论", "Thomas Cormen", "计算机科学", 2009);
    library.addBook("978-0-345678-90-1", "设计模式", "Gang of Four", "软件工程", 1994);
    library.addBook("978-0-456789-01-2", "深入理解计算机系统", "Randal Bryant", "计算机科学", 2010);
    library.addBook("978-0-567890-12-3", "Python编程快速上手", "Al Sweigart", "编程", 2015);

    // 显示所有图书
    library.displayAllBooks();

    // 测试搜索功能
    cout << "\n=== 搜索测试 ===" << endl;

    // 按作者搜索
    auto books_by_author = library.findBooksByAuthor("Bjarne Stroustrup");
    cout << "Bjarne Stroustrup的书:" << endl;
    for (auto* book : books_by_author) {
        book->display();
    }

    // 按分类搜索
    auto programming_books = library.findBooksByCategory("编程");
    cout << "\n编程类图书:" << endl;
    for (auto* book : programming_books) {
        book->display();
    }

    // 按标题关键词搜索
    auto cpp_books = library.searchByTitle("c++");
    cout << "\n包含'C++'的图书:" << endl;
    for (auto* book : cpp_books) {
        book->display();
    }

    // 测试借还书功能
    cout << "\n=== 借还书测试 ===" << endl;
    library.borrowBook("978-0-123456-78-9", "张三");
    library.borrowBook("978-0-123456-78-9", "李四"); // 加入等待队列
    library.borrowBook("978-0-234567-89-0", "张三");

    library.returnBook("978-0-123456-78-9"); // 通知等待者
```

```
// 显示统计信息
library.displayBorrowStatistics();
library.displayWaitingLists();

return 0;
}
```

第六部分：实战练习

6.1 练习1：文本分析器

****目标****：创建一个程序，分析文本文件的各种统计信息。

****要求****：

1. 统计单词频率
2. 找出最长和最短的单词
3. 统计句子数量
4. 找出出现频率最高的字母

```
```cpp
#include <iostream>
#include <fstream>
#include <string>
#include <map>
#include <set>
#include <algorithm>
#include <cctype>
using namespace std;

class TextAnalyzer {
private:
 map<string, int> word_frequency;
 map<char, int> char_frequency;
 set<string> unique_words;
 string longest_word;
 string shortest_word;
 int sentence_count;
 int total_words;

public:
 TextAnalyzer() : sentence_count(0), total_words(0) {}

 void analyzeFile(const string& filename) {
 ifstream file(filename);
 if (!file.is_open()) {
```

```

 cout << "无法打开文件: " << filename << endl;
 return;
}

string line;
while (getline(file, line)) {
 analyzeLine(line);
}

file.close();
}

void analyzeLine(const string& line) {
 string word;
 for (char c : line) {
 if (isalpha(c)) {
 word += tolower(c);
 char_frequency[tolower(c)]++;
 } else {
 if (!word.empty()) {
 processWord(word);
 word.clear();
 }

 if (c == '.' || c == '!' || c == '?') {
 sentence_count++;
 }
 }
 }

 if (!word.empty()) {
 processWord(word);
 }
}

void processWord(const string& word) {
 word_frequency[word]++;
 unique_words.insert(word);
 total_words++;

 if (longest_word.empty() || word.length() > longest_word.length()) {
 longest_word = word;
 }

 if (shortest_word.empty() || word.length() < shortest_word.length()) {
 shortest_word = word;
 }
}

```

```

}

void printReport() {
 cout << "=== 文本分析报告 ===" << endl;
 cout << "总单词数: " << total_words << endl;
 cout << "唯一单词数: " << unique_words.size() << endl;
 cout << "句子数: " << sentence_count << endl;
 cout << "最长单词: " << longest_word << " (" << longest_word.length() << " 字母)" << endl;
 cout << "最短单词: " << shortest_word << " (" << shortest_word.length() << " 字母)" << endl;

 // 最频繁的单词
 auto max_word = max_element(word_frequency.begin(), word_frequency.end(),
 [](const pair<string, int>& a, const pair<string, int>& b) {
 return a.second < b.second;
 });

 if (max_word != word_frequency.end()) {
 cout << "最频繁单词: " << max_word->first
 << " (出现 " << max_word->second << " 次)" << endl;
 }

 // 最频繁的字母
 auto max_char = max_element(char_frequency.begin(), char_frequency.end(),
 [](const pair<char, int>& a, const pair<char, int>& b) {
 return a.second < b.second;
 });

 if (max_char != char_frequency.end()) {
 cout << "最频繁字母: " << max_char->first
 << " (出现 " << max_char->second << " 次)" << endl;
 }

 // 词频前10
 vector<pair<string, int>> sorted_words(word_frequency.begin(), word_frequency.end());
 sort(sorted_words.begin(), sorted_words.end(),
 [](const pair<string, int>& a, const pair<string, int>& b) {
 return a.second > b.second;
 });

 cout << "\n词频前10:" << endl;
 for (int i = 0; i < min(10, (int)sorted_words.size()); ++i) {
 cout << (i+1) << ". " << sorted_words[i].first
 << ": " << sorted_words[i].second << endl;
 }
}
};

```



// 使用示例

```
int main() {
 TextAnalyzer analyzer;

 // 创建示例文件或分析现有文件
 string sample_text = "Hello world! This is a sample text for analysis. "
 "Hello appears twice in this text. "
 "We are analyzing the frequency of words and characters.";

 // 将示例文本写入文件
 ofstream outfile("sample.txt");
 outfile << sample_text;
 outfile.close();

 analyzer.analyzeFile("sample.txt");
 analyzer.printReport();

 return 0;
}
```

## 6.2 练习2：学生成绩管理系统

**目标：**创建一个完整的成绩管理系统，支持多种查询和统计功能。

cpp

```
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <algorithm>
#include <numeric>
#include <iomanip>
using namespace std;

// 学生成绩结构
struct Grade {
 string subject;
 double score;
 int credits;

 Grade(const string& s, double sc, int c) : subject(s), score(sc), credits(c) {}
};

// 学生类
class Student {
public:
 string id;
 string name;
 vector<Grade> grades;

 Student(const string& student_id, const string& student_name)
 : id(student_id), name(student_name) {}

 void addGrade(const string& subject, double score, int credits) {
 grades.emplace_back(subject, score, credits);
 }

 double getGPA() const {
 if (grades.empty()) return 0.0;

 double total_points = 0.0;
 int total_credits = 0;

 for (const auto& grade : grades) {
 total_points += grade.score * grade.credits;
 total_credits += grade.credits;
 }

 return total_credits > 0 ? total_points / total_credits : 0.0;
 }
};
```

```

double getAverageScore() const {
 if (grades.empty()) return 0.0;

 double sum = accumulate(grades.begin(), grades.end(), 0.0,
 [](double acc, const Grade& g) { return acc + g.score; });

 return sum / grades.size();
}

int getTotalCredits() const {
 return accumulate(grades.begin(), grades.end(), 0,
 [](int acc, const Grade& g) { return acc + g.credits; });
}

void display() const {
 cout << "学号: " << id << ", 姓名: " << name << endl;
 cout << "课程成绩:" << endl;

 for (const auto& grade : grades) {
 cout << " " << grade.subject << ": " << grade.score
 << " (" << grade.credits << "学分)" << endl;
 }

 cout << "平均分: " << fixed << setprecision(2) << getAverageScore() << endl;
 cout << "GPA: " << getGPA() << endl;
 cout << "总学分: " << getTotalCredits() << endl << endl;
}
};

```

// 成绩管理系统

```

class GradeManager {
private:
 vector<Student> students;
 map<string, size_t> id_to_index;

public:
 void addStudent(const string& id, const string& name) {
 if (id_to_index.find(id) != id_to_index.end()) {
 cout << "学生已存在: " << id << endl;
 return;
 }

 students.emplace_back(id, name);
 id_to_index[id] = students.size() - 1;
 cout << "添加学生成功: " << name << endl;
 }
}

```

```

void addGrade(const string& id, const string& subject, double score, int credits) {
 auto it = id_to_index.find(id);
 if (it == id_to_index.end()) {
 cout << "找不到学生: " << id << endl;
 return;
 }

 students[it->second].addGrade(subject, score, credits);
 cout << "成绩添加成功" << endl;
}

```

```

void displayStudent(const string& id) {
 auto it = id_to_index.find(id);
 if (it == id_to_index.end()) {
 cout << "找不到学生: " << id << endl;
 return;
 }

 students[it->second].display();
}

```

```

void displayAllStudents() {
 cout << "\n=== 所有学生信息 ===" << endl;
 for (const auto& student : students) {
 student.display();
 }
}

```

```

void displayByGPA() {
 vector<Student> sorted_students = students;
 sort(sorted_students.begin(), sorted_students.end(),
 [](const Student& a, const Student& b) {
 return a.getGPA() > b.getGPA();
 });

 cout << "\n=== 按GPA排序 ===" << endl;
 for (const auto& student : sorted_students) {
 cout << student.name << " (GPA: " << fixed << setprecision(2)
 << student.getGPA() << ")" << endl;
 }
}

```

```

void subjectStatistics(const string& subject) {
 vector<double> scores;

 for (const auto& student : students) {
 for (const auto& grade : student.grades) {

```

```

 if (grade.subject == subject) {
 scores.push_back(grade.score);
 break;
 }
 }
}

if (scores.empty()) {
 cout << "没有找到科目: " << subject << endl;
 return;
}

sort(scores.begin(), scores.end());

double average = accumulate(scores.begin(), scores.end(), 0.0) / scores.size();
double median = scores.size() % 2 == 0 ?
 (scores[scores.size()/2 - 1] + scores[scores.size()/2]) / 2.0 :
 scores[scores.size()/2];

cout << "\n=== " << subject << " 统计信息 ===" << endl;
cout << "参与人数: " << scores.size() << endl;
cout << "平均分: " << fixed << setprecision(2) << average << endl;
cout << "中位数: " << median << endl;
cout << "最高分: " << scores.back() << endl;
cout << "最低分: " << scores.front() << endl;

// 分数分布
map<string, int> distribution;
for (double score : scores) {
 if (score >= 90) distribution["优秀(90-100)"]++;
 else if (score >= 80) distribution["良好(80-89)"]++;
 else if (score >= 70) distribution["中等(70-79)"]++;
 else if (score >= 60) distribution["及格(60-69)"]++;
 else distribution["不及格(<60)"]++;
}

cout << "分数分布:" << endl;
for (const auto& pair : distribution) {
 cout << " " << pair.first << ": " << pair.second << " 人" << endl;
}
}

void findTopStudents(int n = 5) {
 vector<Student> sorted_students = students;
 sort(sorted_students.begin(), sorted_students.end(),
 [](const Student& a, const Student& b) {
 return a.getGPA() > b.getGPA();
 });
}

```

```

 });

 cout << "\n=== 前" << n << "名学生 ===" << endl;
 for (int i = 0; i < min(n, (int)sorted_students.size()); ++i) {
 cout << (i+1) << ". " << sorted_students[i].name
 << " (GPA: " << fixed << setprecision(2)
 << sorted_students[i].getGPA() << ")" << endl;
 }
}
};

```

```

int main() {
 GradeManager manager;

 // 添加学生
 manager.addStudent("2021001", "张三");
 manager.addStudent("2021002", "李四");
 manager.addStudent("2021003", "王五");
 manager.addStudent("2021004", "赵六");

 // 添加成绩
 manager.addGrade("2021001", "数学", 85, 4);
 manager.addGrade("2021001", "英语", 92, 3);
 manager.addGrade("2021001", "物理", 78, 4);

 manager.addGrade("2021002", "数学", 95, 4);
 manager.addGrade("2021002", "英语", 88, 3);
 manager.addGrade("2021002", "物理", 91, 4);

 manager.addGrade("2021003", "数学", 72, 4);
 manager.addGrade("2021003", "英语", 85, 3);
 manager.addGrade("2021003", "物理", 75, 4);

 manager.addGrade("2021004", "数学", 68, 4);
 manager.addGrade("2021004", "英语", 75, 3);
 manager.addGrade("2021004", "物理", 82, 4);

 // 显示各种统计信息
 manager.displayAllStudents();
 manager.displayByGPA();
 manager.findTopStudents(3);
 manager.subjectStatistics("数学");

 return 0;
}

```

### 6.3 练习3：简单的搜索引擎

**目标：**实现一个简单的文档搜索引擎，支持关键词搜索和相关性排序。

cpp

```
#include <iostream>
#include <vector>
#include <map>
#include <set>
#include <string>
#include <algorithm>
#include <sstream>
#include <cmath>
using namespace std;

// 文档类
class Document {
public:
 int id;
 string title;
 string content;
 map<string, int> word_count;
 int total_words;

 Document(int doc_id, const string& doc_title, const string& doc_content)
 : id(doc_id), title(doc_title), content(doc_content), total_words(0) {
 processContent();
 }

private:
 void processContent() {
 stringstream ss(content);
 string word;

 while (ss >> word) {
 // 转换为小写并去掉标点
 string clean_word;
 for (char c : word) {
 if (isalpha(c)) {
 clean_word += tolower(c);
 }
 }

 if (!clean_word.empty() && clean_word.length() > 2) { // 忽略短词
 word_count[clean_word]++;
 total_words++;
 }
 }
 }
};
```



// 搜索结果

```
struct SearchResult {
 int doc_id;
 string title;
 double score;
 vector<string> matched_terms;

 SearchResult(int id, const string& t, double s)
 : doc_id(id), title(t), score(s) {}
};
```

// 搜索引擎

```
class SearchEngine {
private:
 vector<Document> documents;
 map<string, set<int>> inverted_index; // 倒排索引
 map<string, int> document_frequency; // 文档频率

public:
 void addDocument(const string& title, const string& content) {
 int doc_id = documents.size();
 documents.emplace_back(doc_id, title, content);

 // 更新倒排索引
 const Document& doc = documents.back();
 for (const auto& pair : doc.word_count) {
 inverted_index[pair.first].insert(doc_id);
 if (pair.second > 0) {
 document_frequency[pair.first]++;
 }
 }

 cout << "文档添加成功: " << title << endl;
 }

 vector<SearchResult> search(const string& query, int max_results = 10) {
 vector<string> query_terms = parseQuery(query);
 map<int, double> doc_scores;
 map<int, set<string>> doc_matched_terms;

 // 计算每个文档的得分
 for (const string& term : query_terms) {
 auto it = inverted_index.find(term);
 if (it != inverted_index.end()) {
 double idf = calculateIDF(term);

 for (int doc_id : it->second) {
```

```

 double tf = calculateTF(doc_id, term);
 double tfidf = tf * idf;

 doc_scores[doc_id] += tfidf;
 doc_matched_terms[doc_id].insert(term);
 }
}

// 创建搜索结果
vector<SearchResult> results;
for (const auto& pair : doc_scores) {
 int doc_id = pair.first;
 double score = pair.second;

 SearchResult result(doc_id, documents[doc_id].title, score);
 for (const string& term : doc_matched_terms[doc_id]) {
 result.matched_terms.push_back(term);
 }

 results.push_back(result);
}

// 按得分排序
sort(results.begin(), results.end(),
 [](const SearchResult& a, const SearchResult& b) {
 return a.score > b.score;
 });

// 限制结果数量
if (results.size() > max_results) {
 results.resize(max_results);
}

return results;
}

void displaySearchResults(const vector<SearchResult>& results, const string& query) {
 cout << "\n搜索 \"" << query << "\" 的结果 (" << results.size() << " 个):" << endl;
 cout << string(50, '-') << endl;

 for (size_t i = 0; i < results.size(); ++i) {
 cout << (i+1) << ". " << results[i].title << endl;
 cout << " 得分: " << fixed << setprecision(4) << results[i].score << endl;
 cout << " 匹配词: ";
 for (const string& term : results[i].matched_terms) {
 cout << term << " ";
 }
 }
}

```

```

 }
 cout << endl;

 // 显示内容片段
 string snippet = getSnippet(results[i].doc_id, results[i].matched_terms);
 cout << " 摘要: " << snippet << endl << endl;
}
}

```

private:

```

vector<string> parseQuery(const string& query) {
 vector<string> terms;
 stringstream ss(query);
 string word;

 while (ss >> word) {
 string clean_word;
 for (char c : word) {
 if (isalpha(c)) {
 clean_word += tolower(c);
 }
 }

 if (!clean_word.empty() && clean_word.length() > 2) {
 terms.push_back(clean_word);
 }
 }

 return terms;
}

double calculateTF(int doc_id, const string& term) {
 const Document& doc = documents[doc_id];
 auto it = doc.word_count.find(term);
 if (it == doc.word_count.end()) {
 return 0.0;
 }

 // TF = (词频 / 文档总词数)
 return static_cast<double>(it->second) / doc.total_words;
}

double calculateIDF(const string& term) {
 auto it = document_frequency.find(term);
 if (it == document_frequency.end()) {
 return 0.0;
 }
}

```

```

// IDF = log(总文档数 / 包含该词的文档数)
return log(static_cast<double>(documents.size()) / it->second);
}

string getSnippet(int doc_id, const vector<string>& matched_terms) {
 const string& content = documents[doc_id].content;

 // 简单实现：返回包含匹配词的第一个句子
 size_t start = 0;
 for (const string& term : matched_terms) {
 size_t pos = content.find(term);
 if (pos != string::npos) {
 // 找到句子的开始
 while (start > 0 && content[start-1] != '.' && content[start-1] != '!' && content[start-1] != '?') {
 start--;
 }

 // 找到句子的结束
 size_t end = pos;
 while (end < content.length() && content[end] != '.' && content[end] != '!' && content[end] != '?') {
 end++;
 }

 if (end < content.length()) end++;

 string snippet = content.substr(start, min(end - start, static_cast<size_t>(100)));
 return snippet + (end - start > 100 ? "... : ";

 }
 }

 // 如果没找到匹配词，返回前100个字符
 return content.substr(0, min(static_cast<size_t>(100), content.length())) +
 (content.length() > 100 ? "... : ";

};

int main() {
 SearchEngine engine;

 // 添加一些示例文档
 engine.addDocument(
 "C++编程入门",
 "C++是一种通用的编程语言，支持面向对象编程。它是C语言的扩展，"
 "提供了类、继承、多态等面向对象特性。C++广泛应用于系统编程、"
 "游戏开发、嵌入式系统等领域。"
);
};

```

```
engine.addDocument(
 "Python编程基础",
 "Python是一种高级编程语言，以其简洁的语法而闻名。Python支持"
 "多种编程范式，包括面向对象编程、函数式编程等。它广泛应用于"
 "数据科学、机器学习、网络开发等领域。"
);

engine.addDocument(
 "Java编程指南",
 "Java是一种面向对象的编程语言，具有跨平台# C++ string类和STL标准模板库自学手册
```

\*初学者完整指南\*

---

## 目录

- 1. [引言与学习目标](#引言与学习目标)
- 2. [第一部分：string类详解](#第一部分string类详解)
- 3. [第二部分：智能指针](#第二部分智能指针)
- 4. [第三部分：STL标准模板库](#第三部分stl标准模板库)
- 5. [第四部分：泛型编程概念](#第四部分泛型编程概念)
- 6. [第五部分：函数对象与算法](#第五部分函数对象与算法)
- 7. [第六部分：实战练习](#第六部分实战练习)
- 8. [附录：快速参考](#附录快速参考)

---

## 引言与学习目标

### 为什么学习string类和STL？

在学习C++的过程中，您可能已经接触过C风格的字符串（char数组）和基本的内存管理。但是，现代C++提供了更强

- **string类**：让字符串操作变得简单安全
- **智能指针**：自动管理内存，避免内存泄漏
- **STL容器**：提供各种数据结构（向量、列表、集合等）
- **算法库**：提供排序、查找等通用算法

### 学习路径建议

建议您按照以下顺序学习：

- 1. 先掌握string类的基本使用
- 2. 理解智能指针的概念和用法
- 3. 学习STL容器的特点和选择
- 4. 掌握迭代器的使用

5. 学习常用算法
6. 通过实战练习巩固知识

---

## ## 第一部分：string类详解

### ### 1.1 什么是string类？

想象一下，如果您要在程序中处理文本，使用C风格字符串会遇到很多问题：

- 需要手动管理内存
- 容易发生缓冲区溢出
- 字符串操作复杂

string类就像是一个"智能的字符串管理器"，它帮您处理这些复杂的细节。

```
```cpp
// C风格字符串的问题
char oldStr[100]; // 固定大小，可能不够用
strcpy(oldStr, "Hello"); // 需要记住使用strcpy
strcat(oldStr, " World"); // 可能溢出

// string类的优势
string newStr = "Hello"; // 自动分配内存
newStr += " World";      // 简单安全的操作
```

1.2 string类的构造方法

string类提供了多种创建字符串的方式，就像有不同的工具来完成同一个任务：

```
cpp
```

```
#include <string>
using namespace std;

// 方法1：默认构造（空字符串）
string str1; // 创建空字符串

// 方法2：从C风格字符串构造
string str2("Hello World");

// 方法3：复制构造
string str3(str2); // str3 = "Hello World"

// 方法4：重复字符构造
string str4(5, 'A'); // str4 = "AAAAA"

// 方法5：从部分字符串构造
string str5("Hello World", 5); // str5 = "Hello"

// 方法6：从另一个string的子串构造
string str6(str2, 6, 5); // 从位置6开始，取5个字符 = "World"
```

理解要点：每种构造方法都有其特定的使用场景。默认构造用于稍后赋值，C风格字符串构造最常用，重复字符构造适合创建分隔线等。

1.3 string类的输入输出

string类的输入输出比C风格字符串更安全、更方便：

```
cpp
```

```

#include <iostream>
#include <string>
using namespace std;

int main() {
    string name;

    // 基本输入输出
    cout << "请输入您的姓名: ";
    cin >> name; // 读取一个单词 (遇到空格停止)
    cout << "您好, " << name << endl;

    // 读取整行 (包括空格)
    string fullName;
    cout << "请输入您的全名: ";
    getline(cin, fullName); // 读取整行
    cout << "您的全名是: " << fullName << endl;

    return 0;
}

```

关键差异：

- `cin >> str`：读取单词（遇空格、换行符停止）
- `getline(cin, str)`：读取整行（遇换行符停止）

1.4 string类的常用操作

字符串连接和修改

```

cpp

string str1 = "Hello";
string str2 = " World";

// 连接字符串
string result = str1 + str2; // "Hello World"
str1 += str2; // str1 变成 "Hello World"

// 插入和删除
str1.insert(5, ","); // 在位置5插入逗号: "Hello, World"
str1.erase(5, 1);    // 从位置5删除1个字符

// 替换
str1.replace(0, 5, "Hi"); // 将前5个字符替换为"Hi"

```


字符串查找

```
cpp

string text = "Hello World Hello";

// 查找子字符串
size_t pos = text.find("World"); // 返回6
if (pos != string::npos) {
    cout << "找到'World'在位置: " << pos << endl;
}

// 查找字符
pos = text.find('o'); // 返回4 (第一个'o'的位置)

// 反向查找
pos = text.rfind("Hello"); // 返回12 (最后一个"Hello"的位置)
```

重要概念： `string::npos` 是一个特殊值，表示"未找到"。它等于string能存储的最大字符数。

字符串比较

```
cpp

string str1 = "apple";
string str2 = "banana";

// 直接比较 (按字典序)
if (str1 < str2) {
    cout << str1 << "在字典中排在" << str2 << "前面" << endl;
}

// 相等比较
if (str1 == str2) {
    cout << "字符串相等" << endl;
}
```

1.5 实战示例：简单的单词游戏

让我们通过一个实际例子来巩固string类的使用：

```
cpp
```

```
#include <iostream>
#include <string>
#include <algorithm> // 用于transform函数
#include <cctype>     // 用于tolower函数
using namespace std;

// 将字符串转换为小写
string toLower(string str) {
    transform(str.begin(), str.end(), str.begin(), ::tolower);
    return str;
}

int main() {
    string secretWord = "programming";
    string guess;
    int attempts = 0;
    const int maxAttempts = 5;

    cout << "=== 猜单词游戏 ===" << endl;
    cout << "我想了一个" << secretWord.length() << "个字母的单词" << endl;
    cout << "您有" << maxAttempts << "次机会猜测" << endl;

    while (attempts < maxAttempts) {
        cout << "\n请输入您的猜测: ";
        getline(cin, guess);
        attempts++;

        // 转换为小写进行比较
        string lowerGuess = toLower(guess);
        string lowerSecret = toLower(secretWord);

        if (lowerGuess == lowerSecret) {
            cout << "恭喜！您猜对了！" << endl;
            cout << "您用了" << attempts << "次尝试" << endl;
            return 0;
        }

        // 给出提示
        if (lowerGuess < lowerSecret) {
            cout << "您的猜测在字典中排在答案前面" << endl;
        } else {
            cout << "您的猜测在字典中排在答案后面" << endl;
        }

        cout << "还剩" << (maxAttempts - attempts) << "次机会" << endl;
    }
}
```

```
cout << "\n游戏结束! 答案是: " << secretWord << endl;
return 0;
}
```

学习要点:

1. 使用 `length()` 获取字符串长度
2. 使用 `getline()` 读取包含空格的输入
3. 使用 `transform()` 和 `tolower()` 进行大小写转换
4. 字符串比较的字典序规则

第二部分：智能指针

2.1 为什么需要智能指针？

在传统的C++中，动态内存管理是一个容易出错的地方：

```
cpp
// 传统指针的问题
void problematicFunction() {
    int* ptr = new int(42); // 分配内存

    // 如果这里发生异常或提前返回
    if (someCondition) {
        return; // 忘记delete, 造成内存泄漏!
    }

    delete ptr; // 可能永远执行不到
}
```

智能指针就像是一个"自动管家"，当它们离开作用域时，会自动清理内存：

```
cpp
```

```

#include <memory>

void smartFunction() {
    unique_ptr<int> ptr(new int(42)); // 智能指针

    // 无论函数如何退出，内存都会被自动释放
    if (someCondition) {
        return; // 安全！智能指针的析构函数会自动delete
    }

    // 函数结束时也会自动释放
}

```

2.2 三种智能指针的特点

unique_ptr：独占所有权

unique_ptr 就像一个“独占的管家”，同一时间只能有一个 unique_ptr 拥有某个对象：

```

cpp

#include <memory>
#include <iostream>
using namespace std;

int main() {
    // 创建unique_ptr
    unique_ptr<int> ptr1(new int(42));
    // 更好的创建方式 (C++14推荐)
    auto ptr2 = make_unique<int>(100);

    cout << "ptr1指向的值: " << *ptr1 << endl;
    cout << "ptr2指向的值: " << *ptr2 << endl;

    // 转移所有权
    unique_ptr<int> ptr3 = move(ptr1); // ptr1变为空，ptr3获得所有权

    if (ptr1 == nullptr) {
        cout << "ptr1现在是空的" << endl;
    }

    cout << "ptr3指向的值: " << *ptr3 << endl;

    return 0; // 所有智能指针自动释放内存
}

```

关键特性：

- 不能拷贝，只能移动
- 离开作用域时自动delete
- 支持数组：`unique_ptr<int[]> arr(new int[10]);`

shared_ptr：共享所有权

`shared_ptr`像是"共享的管家"，多个`shared_ptr`可以指向同一个对象，只有当最后一个`shared_ptr`销毁时，对象才会被删除：

```
cpp

#include <memory>
#include <iostream>
using namespace std;

int main() {
    // 创建shared_ptr
    shared_ptr<int> ptr1 = make_shared<int>(42);
    cout << "引用计数: " << ptr1.use_count() << endl; // 输出: 1

    {
        shared_ptr<int> ptr2 = ptr1; // 共享所有权
        cout << "引用计数: " << ptr1.use_count() << endl; // 输出: 2
        cout << "ptr2指向的值: " << *ptr2 << endl;
    } // ptr2离开作用域，引用计数减1

    cout << "引用计数: " << ptr1.use_count() << endl; // 输出: 1

    return 0; // ptr1离开作用域，引用计数变为0，对象被删除
}
```

weak_ptr：观察者

`weak_ptr`像是"观察者"，它可以观察`shared_ptr`管理的对象，但不影响引用计数：

```
cpp
```

```

#include <memory>
#include <iostream>
using namespace std;

int main() {
    shared_ptr<int> shared = make_shared<int>(42);
    weak_ptr<int> weak = shared; // 不增加引用计数

    cout << "shared引用计数: " << shared.use_count() << endl; // 仍然是1

    // 使用weak_ptr前需要检查对象是否还存在
    if (auto locked = weak.lock()) { // 尝试获取shared_ptr
        cout << "对象还存在, 值为: " << *locked << endl;
    } else {
        cout << "对象已被删除" << endl;
    }

    shared.reset(); // 删除shared_ptr

    if (auto locked = weak.lock()) {
        cout << "对象还存在" << endl;
    } else {
        cout << "对象已被删除" << endl; // 这行会执行
    }

    return 0;
}

```

2.3 实战示例：资源管理系统

让我们通过一个文件管理系统的例子来看智能指针的实际应用：

cpp

```
#include <memory>
#include <string>
#include <vector>
#include <iostream>
using namespace std;

// 文件类
class File {
private:
    string filename;
    string content;

public:
    File(const string& name) : filename(name) {
        cout << "打开文件: " << filename << endl;
    }

    ~File() {
        cout << "关闭文件: " << filename << endl;
    }

    void write(const string& data) {
        content += data;
        cout << "写入文件 " << filename << ": " << data << endl;
    }

    string getFilename() const { return filename; }
};

// 文档管理器
class DocumentManager {
private:
    vector<shared_ptr<File>> files;

public:
    // 创建新文件
    shared_ptr<File> createFile(const string& filename) {
        auto file = make_shared<File>(filename);
        files.push_back(file);
        return file;
    }

    // 获取文件
    shared_ptr<File> getFile(const string& filename) {
        for (auto& file : files) {
            if (file->getFilename() == filename) {
```

```

        return file; // 返回shared_ptr，增加引用计数
    }
}
return nullptr;
}

void listFiles() {
    cout << "\n当前打开的文件:" << endl;
    for (auto& file : files) {
        cout << "- " << file->getFilename()
            << " (引用计数: " << file.use_count() << ")" << endl;
    }
}
};

int main() {
    DocumentManager manager;

    // 创建文件
    auto file1 = manager.createFile("document1.txt");
    auto file2 = manager.createFile("document2.txt");

    // 写入数据
    file1->write("Hello ");
    file1->write("World!");

    // 获取已存在的文件（增加引用计数）
    auto file1_copy = manager.getFile("document1.txt");
    file1_copy->write(" Additional content.");

    manager.listFiles();

    // 释放一个引用
    file1.reset();
    cout << "\n释放file1后:" << endl;
    manager.listFiles();

    // 当所有shared_ptr都销毁时，文件会自动关闭
    cout << "\n程序结束，所有文件将自动关闭:" << endl;

    return 0;
}

```

学习要点：

1. `make_shared` 比直接使用 `new` 更高效

2. `shared_ptr`可以安全地在容器中存储和传递
 3. 引用计数机制确保资源在合适的时候释放
 4. `reset()`可以主动释放智能指针
-

第三部分：STL标准模板库

3.1 什么是STL?

STL (Standard Template Library) 就像是一个"程序员的工具箱", 里面包含了各种经过精心设计和优化的数据结构和算法。

STL的四大组件:

1. **容器 (Containers)**: 存储数据的结构, 如vector、list、set
2. **迭代器 (Iterators)**: 遍历容器的"指针"
3. **算法 (Algorithms)**: 处理数据的函数, 如sort、find
4. **函数对象 (Function Objects)**: 可以像函数一样调用的对象

3.2 vector: 动态数组

`vector`就像一个可以自动扩容的数组:

```
cpp
```

```

#include <vector>
#include <iostream>
using namespace std;

int main() {
    // 创建vector
    vector<int> numbers; // 空vector
    vector<int> scores(5, 100); // 5个元素，都是100
    vector<string> names{"Alice", "Bob", "Charlie"}; // 初始化列表

    // 添加元素
    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    // 访问元素
    cout << "第一个元素: " << numbers[0] << endl;
    cout << "第二个元素: " << numbers.at(1) << endl; // at()会检查边界

    // 遍历vector
    cout << "所有元素: ";
    for (size_t i = 0; i < numbers.size(); ++i) {
        cout << numbers[i] << " ";
    }
    cout << endl;

    // 范围for循环 (C++11)
    cout << "使用范围for: ";
    for (const auto& num : numbers) {
        cout << num << " ";
    }
    cout << endl;

    // 插入和删除
    numbers.insert(numbers.begin() + 1, 15); // 在位置1插入15
    numbers.erase(numbers.begin()); // 删除第一个元素

    // 检查容量和大小
    cout << "大小: " << numbers.size() << endl;
    cout << "容量: " << numbers.capacity() << endl;

    return 0;
}

```

vector的特点：

- 随机访问： $O(1)$ 时间访问任意元素
- 尾部插入： $O(1)$ 平均时间
- 中间插入/删除： $O(n)$ 时间
- 自动内存管理

3.3 list：双向链表

list 像是一串链接的节点，适合频繁插入和删除：

```
cpp
```

```

#include <list>
#include <iostream>
using namespace std;

int main() {
    list<int> numbers{1, 2, 3, 4, 5};

    // 在开头和结尾添加元素
    numbers.push_front(0); // 在开头插入0
    numbers.push_back(6); // 在结尾插入6

    // 在中间插入
    auto it = numbers.begin();
    advance(it, 3); // 移动到第4个位置
    numbers.insert(it, 99); // 插入99

    // 遍历
    cout << "链表内容: ";
    for (const auto& num : numbers) {
        cout << num << " ";
    }
    cout << endl;

    // 删除特定值
    numbers.remove(99); // 删除所有值为99的元素

    // 排序 (list有自己的sort方法)
    numbers.sort();

    cout << "排序后: ";
    for (const auto& num : numbers) {
        cout << num << " ";
    }
    cout << endl;

    return 0;
}

```

list的特点：

- 任意位置插入/删除：O(1)时间
- 不支持随机访问：不能使用[]操作符
- 双向遍历：支持前向和后向迭代器

3.4 map：关联容器

map 就像一个字典，存储键值对：

cpp

```
#include <map>
#include <string>
#include <iostream>
using namespace std;

int main() {
    // 创建学生成绩映射
    map<string, int> grades;

    // 添加数据
    grades["Alice"] = 95;
    grades["Bob"] = 87;
    grades["Charlie"] = 92;

    // 另一种插入方式
    grades.insert(make_pair("David", 88));
    grades.insert({"Eve", 96}); // C++11 语法

    // 访问数据
    cout << "Alice的成绩: " << grades["Alice"] << endl;

    // 安全访问（如果键不存在不会创建新元素）
    auto it = grades.find("Frank");
    if (it != grades.end()) {
        cout << "Frank的成绩: " << it->second << endl;
    } else {
        cout << "没找到Frank的成绩" << endl;
    }

    // 遍历map
    cout << "\n所有学生成绩:" << endl;
    for (const auto& pair : grades) {
        cout << pair.first << ": " << pair.second << endl;
    }

    // 统计字符出现次数的例子
    string text = "hello world";
    map<char, int> charCount;

    for (char c : text) {
        if (c != ' ') { // 忽略空格
            charCount[c]++; // 如果键不存在，会自动创建并初始化为0
        }
    }

    cout << "\n字符出现次数:" << endl;
```

```
for (const auto& pair : charCount) {  
    cout << "" << pair.first << ": " << pair.second << endl;  
}  
  
return 0;  
}
```

map的特点：

- 键唯一：每个键只能出现一次
- 自动排序：按键的顺序排序
- 查找效率： $O(\log n)$ 时间复杂度

3.5 set：集合容器

set 就像一个不允许重复元素的有序集合：

cpp

```
#include <set>
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    // 创建集合
    set<int> numbers{3, 1, 4, 1, 5, 9, 2, 6}; // 重复的1会被忽略

    // 插入元素
    numbers.insert(7);
    numbers.insert(3); // 重复元素，不会插入

    // 遍历集合（自动排序）
    cout << "集合内容: ";
    for (const auto& num : numbers) {
        cout << num << " ";
    }
    cout << endl;

    // 查找元素
    if (numbers.find(5) != numbers.end()) {
        cout << "找到了5" << endl;
    }

    // 删除元素
    numbers.erase(4);

    // 集合运算示例
    set<int> set1{1, 2, 3, 4, 5};
    set<int> set2{4, 5, 6, 7, 8};
    set<int> result;

    // 并集
    set_union(set1.begin(), set1.end(),
              set2.begin(), set2.end(),
              inserter(result, result.begin()));

    cout << "并集: ";
    for (const auto& num : result) {
        cout << num << " ";
    }
    cout << endl;

    // 交集
    result.clear();
```



```
set_intersection(set1.begin(), set1.end(),
                set2.begin(), set2.end(),
                inserter(result, result.begin()));

cout << "交集: ";
for (const auto& num : result) {
    cout << num << " ";
}
cout << endl;

return 0;
}
```

3.6 实战示例：学生管理系统

让我们综合运用多种STL容器来创建一个简单的学生管理系统：

cpp

```
#include <iostream>
#include <vector>
#include <map>
#include <set>
#include <string>
#include <algorithm>
using namespace std;

// 学生类
class Student {
public:
    string name;
    int age;
    vector<int> grades;

    Student(const string& n, int a) : name(n), age(a) {}

    void addGrade(int grade) {
        grades.push_back(grade);
    }

    double getAverage() const {
        if (grades.empty()) return 0.0;
        int sum = 0;
        for (int grade : grades) {
            sum += grade;
        }
        return static_cast<double>(sum) / grades.size();
    }

    void display() const {
        cout << "姓名: " << name << ", 年龄: " << age
              << ", 平均分: " << getAverage() << endl;
    }
};

// 学生管理系统类
class StudentManager {
private:
    vector<Student> students;    // 存储所有学生
    map<string, int> nameToIndex; // 姓名到索引的映射

public:
    // 添加学生
    void addStudent(const string& name, int age) {
        if (nameToIndex.find(name) != nameToIndex.end()) {
```

```

        cout << "学生 " << name << " 已存在! " << endl;
        return;
    }

    students.emplace_back(name, age);
    nameToIndex[name] = students.size() - 1;
    cout << "成功添加学生: " << name << endl;
}

// 为学生添加成绩
void addGrade(const string& name, int grade) {
    auto it = nameToIndex.find(name);
    if (it == nameToIndex.end()) {
        cout << "找不到学生: " << name << endl;
        return;
    }

    students[it->second].addGrade(grade);
    cout << "为 " << name << " 添加成绩: " << grade << endl;
}

// 显示所有学生
void displayAllStudents() const {
    cout << "\n=== 所有学生信息 ===" << endl;
    for (const auto& student : students) {
        student.display();
    }
}

// 按平均分排序显示
void displayByAverage() const {
    vector<Student> sortedStudents = students;
    sort(sortedStudents.begin(), sortedStudents.end(),
        [](const Student& a, const Student& b) {
            return a.getAverage() > b.getAverage();
        });

    cout << "\n=== 按平均分排序 ===" << endl;
    for (const auto& student : sortedStudents) {
        student.display();
    }
}

// 统计成绩分布
void analyzeGrades() const {
    map<char, int> gradeDistribution; // A, B, C, D, F

```

```

for (const auto& student : students) {
    double avg = student.getAverage();
    char grade;
    if (avg >= 90) grade = 'A';
    else if (avg >= 80) grade = 'B';
    else if (avg >= 70) grade = 'C';
    else if (avg >= 60) grade = 'D';
    else grade = 'F';

    gradeDistribution[grade]++;
}

cout << "\n=== 成绩分布统计 ===" << endl;
for (const auto& pair : gradeDistribution) {
    cout << "等级 " << pair.first << ": " << pair.second << " 人" << endl;
}
}
};

```

```

int main() {
    StudentManager manager;

    // 添加学生
    manager.addStudent("张三", 20);
    manager.addStudent("李四", 19);
    manager.addStudent("王五", 21);
    manager.addStudent("赵六", 20);

    // 添加成绩
    manager.addGrade("张三", 85);
    manager.addGrade("张三", 92);
    manager.addGrade("张三", 78);

    manager.addGrade("李四", 95);
    manager.addGrade("李四", 88);
    manager.addGrade("李四", 91);

    manager.addGrade("王五", 72);
    manager.addGrade("王五", 69);
    manager.addGrade("王五", 75);

    manager.addGrade("赵六", 58);
    manager.addGrade("赵六", 62);
    manager.addGrade("赵六", 55);

    // 显示信息
    manager.displayAllStudents();
}

```

```
manager.displayByAverage();
```

```
manager.analyzeGrades();
```

```
return 0;
```

```
}
```